UNIT 21 100 min

Training a PyTorch Model

Turn a dataset into tensors, choose a loss and optimizer, run the training loop, evaluate honestly, and save the model.

By the end of this unit you will be able to

- Prepare a dataset as tensors with a train/test split and feature scaling
- Choose a loss function for regression and for classification
- Write the five steps of a training loop: zero gradients, forward, loss, backward, step
- Evaluate a model with `model.eval()` and `torch.no_grad()`
- Save and reload a model with `state_dict`

Note

Where to run this unit. Use Google Colab or a local install with `pip install torch scikit-learn` . PyTorch does not run in the browser sandbox. The code uses a fixed random seed so results repeat closely from run to run; exact decimals can vary slightly between machines. **Run the code blocks in order in one notebook.** Each block uses variables created by the blocks above it.

The training loop

Training is a loop. Each pass over the training data is an **epoch**. For each batch, the model makes predictions, a **loss** measures how wrong they are, and the optimizer nudges the weights to reduce that loss. Everything else in this unit fills in those steps.

1. Clear old gradients: `optimizer.zero_grad()`
2. Forward pass: `logits = model(x)`
3. Compute the loss: `loss = loss_fn(logits, y)`
4. Backward pass: `loss.backward()`
5. Update the weights: `optimizer.step()`

Prepare the data

Start from the iris data used in Unit 19. Split it first, then scale using statistics from the training set only (the same rule as the pipeline in Unit 19). Convert the NumPy arrays to tensors. Classification labels must be `int64` for `CrossEntropyLoss`.

data.py

```
import torch
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

iris = load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    iris.data, iris.target, test_size=0.25, random_state=42, stratify=iris.targ
)

scaler = StandardScaler().fit(X_train)          # learn from training data only
X_train = torch.tensor(scaler.transform(X_train), dtype=torch.float32)
X_test = torch.tensor(scaler.transform(X_test), dtype=torch.float32)
y_train = torch.tensor(y_train, dtype=torch.long)
y_test = torch.tensor(y_test, dtype=torch.long)

print(X_train.shape, y_train.dtype)
```

> **Note**
> `stratify=iris.target` keeps the same proportion of each species in both sets. It matters when classes are small or uneven.

Model, loss, and optimizer

- **Model:** a small network that maps 4 features to 3 class scores.
- **Loss:** `CrossEntropyLoss` for classification. It takes raw logits and the integer class labels. For regression, `MSELoss` is the usual choice.
- **Optimizer:** `Adam` adjusts each weight using its gradient history. Its learning rate (`lr`) sets the step size. Too large and the loss bounces; too small and training is slow.

setup.py

```
import torch
import torch.nn as nn

torch.manual_seed(0)

model = nn.Sequential(
    nn.Linear(4, 16),
    nn.ReLU(),
    nn.Linear(16, 3)
)
loss_fn = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=0.01)
```

Running the loop

train.py

```
for epoch in range(200):
    model.train()
    optimizer.zero_grad()
    logits = model(X_train)
    loss = loss_fn(logits, y_train)
    loss.backward()
    optimizer.step()

    if (epoch + 1) % 50 == 0:
        print(f"epoch {epoch + 1:3d}  loss {loss.item():.3f}")
```

This loop runs the whole training set through the model at once, which is called full-batch training. Real datasets are too large for that, so they are split into **mini-batches**, usually fetched with a `DataLoader`. The five steps stay the same.

> **Watch out**
> Forgetting `optimizer.zero_grad()` makes gradients add up across steps, and training behaves strangely. Put it first in the loop every time.

Evaluate honestly

Switch to evaluation mode with `model.eval()`, which turns off behaviors such as dropout that only apply during training. Wrap the calculation in `torch.no_grad()` so PyTorch does not build the graph needed for gradients. Score on the test set, which the model has never used for training.

evaluate.py

```
model.eval()
with torch.no_grad():
    test_logits = model(X_test)
    predicted = test_logits.argmax(dim=1)
    accuracy = (predicted == y_test).float().mean().item()

print(f"test accuracy: {accuracy:.2f}")
print("above 0.9:", accuracy > 0.9)
```

`argmax(dim=1)` picks the highest score in each row. Comparing with the labels and taking the mean gives the fraction correct.

Save and reload

Save the **state_dict**, which is a dictionary of the learned weights, rather than the whole model object. To reload, build the same architecture first, then load the weights into it.

save_load.py

```
import torch
import torch.nn as nn

model = nn.Sequential(nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, 3))
torch.save(model.state_dict(), "iris_model.pt")

fresh = nn.Sequential(nn.Linear(4, 16), nn.ReLU(), nn.Linear(16, 3))
fresh.load_state_dict(torch.load("iris_model.pt"))
fresh.eval()
print("reloaded")
```

Tip
Newer PyTorch versions may warn about `weights_only` when loading. Passing `weights_only=True` to `torch.load` is the safe choice for a state_dict.

Common problems

Symptom	Likely cause	Fix
Loss is `nan`	Learning rate too high, or bad input values	Lower `lr`; check scaling
Loss does not go down	Learning rate too low, or gradients not flowing	Raise `lr`; check `backward()` and `step()`
Training accuracy high, test accuracy low	Overfitting	More data, smaller model, early stopping
Shape or dtype error	Labels not `long`, or features not `float32`	Cast with `.long()` or `.float()`

Reason before you run

- Without `optimizer.zero_grad()`, what happens to `.grad` between steps? *It accumulates, because PyTorch adds each new backward pass to the old gradients.*

- Why evaluate with `torch.no_grad()`? *No gradients are needed for predictions, so it saves memory and time.*
- Why fit the scaler on the training set only? *So the test set stays unseen, as in Unit 19.*

Practice

▼ Exercise 1 — Fit a line

Train a single `nn.Linear(1, 1)` with `MSELoss` and SGD (`lr=0.05`) to learn the rule `y = 2x + 1` from `x = torch.linspace(0, 1, 20).reshape(-1, 1)`. Train for 500 epochs, then print the learned weight and bias, rounded to one decimal place.

▼ Hint

Build `y = 2 * x + 1` as the target. Then the loop is the same five steps as above.

Discussion

1. Why do we keep a test set that the model never trains on, even when the data is small?
2. The loss went down, but the test accuracy did not improve. List three things you would check, in order.
3. Would you deploy a model that scores 0.95 on a test set you looked at a hundred times while tuning? Why or why not?

Key terms

Epoch

One full pass of the training data through the model.

Batch / mini-batch

A subset of the training data processed together in one step of the loop.

Loss function

A formula that measures how wrong the predictions are. Training minimizes it.

CrossEntropyLoss

The standard loss for classification. It takes raw logits and integer class labels.

Optimizer

The algorithm that updates the weights from their gradients, such as SGD or Adam.

Learning rate

The size of each update step taken by the optimizer.

model.eval()

Switches a model to evaluation behavior, turning off training-only behaviors such as dropout.

torch.no_grad()

A context manager that turns off gradient tracking for faster, lighter evaluation.

state_dict

A dictionary of a model's learned parameters, used to save and load it.

Stratify

Splitting data so each class keeps the same proportion in the training and test sets.

See all terms in the glossary →